Manual Técnico
Módulo Herrajes

Versión 1.0 | 13/05/2026

Documentación técnica para desarrolladores y administradores del sistema.
Arquitectura, base de datos, dependencias y procesos internos.

Índice de Contenido

#	Sección
1.	Arquitectura del Sistema
2.	Estructura de Archivos
3.	Base de Datos
3.1	Esquema de la Tabla
3.2	Procedimientos SQL
4.	Dependencias
5.	Módulos y Clases Principales
5.1	pantalla_herrajes.py
5.2	db_manager.py
5.3	Parsers de Importación
5.4	ventana_divisora.py
5.5	imprimir.py
5.6	utils.py
6.	Flujo de Importación
7.	Hilos y Concurrencia
8.	Manejo de Errores
9.	Configuración y Personalización

1. Arquitectura del Sistema

El Módulo de Herrajes sigue una arquitectura de **tres capas** dentro del ecosistema ERP VAH:

[Imagen pendiente: arquitectura_sistema.png]

Capa	Tecnología	Responsabilidad
Presentación	PyQt6	Interfaz gráfica, validación de entrada, UX
Lógica de Negocio	Python 3.12	Parsers, cálculos de precios, workers, PDFs
Acceso a Datos	pyodbc + SQL Server	ODBC, CRUD, MERGE masivo, transacciones
Almacenamiento	SQL Server	Tabla materiales_herraje

2. Estructura de Archivos

[Imagen pendiente: estructura_modulos.png]

Archivo	Tipo	Descripción
pantalla_herrajes.py	Principal	UI principal, worker de importación, diálogos
db_manager.py	Datos	DBManager: conexión, CRUD, bulk upsert con MERGE
imprimir.py	Utilidad	PDF con ReportLab (catálogo de materiales)
utils.py	Utilidad	dec2, detección columnas, ordenamiento
ventana_divisora.py	Interfaz	Diálogo para cantidad divisora en importación
parsers/__init__.py	Paquete	Re-exporta parsers
parsers/parse_csv.py	Parser	CSV genérico
parsers/parse_excel_herralum.py	Parser	Herralum: detección, precio unitario, factor RL
parsers/parse_excel_profilo.py	Parser	Profilo: columnas fijas, limpieza precios

3. Base de Datos

3.1 Esquema de la Tabla

[Imagen pendiente: diagrama_bd.png]

Columna	Tipo	Restricciones	Descripción
Clave	VARCHAR(20)	PK, NOT NULL	Código único del producto
Descripcion	VARCHAR(100)	NULL	Nombre del herraje
Precio	DECIMAL(10,2)	NULL	Precio unitario (2 decimales)

```
CREATE TABLE dbo.materiales_herraje (
    Clave       VARCHAR(20)   NOT NULL PRIMARY KEY,
    Descripcion VARCHAR(100)  NULL,
    Precio      DECIMAL(10,2) NULL
);
```

3.2 Procedimientos SQL Principales

El módulo usa SQL dinámico desde Python vía pyodbc. Operación principal: **MERGE masivo** con tabla temporal:

```
MERGE dbo.materiales_herraje AS T
USING #imp2 AS S ON T.Clave = S.Clave
WHEN MATCHED THEN
  UPDATE SET
    T.Descripcion = CASE WHEN S.Descripcion IS NOT NULL
      THEN S.Descripcion ELSE T.Descripcion END,
    T.Precio = CASE
      WHEN T.Precio IS NULL THEN S.Precio
      WHEN S.Precio > T.Precio THEN S.Precio
      ELSE T.Precio END
WHEN NOT MATCHED BY TARGET THEN
  INSERT (Clave, Descripcion, Precio)
  VALUES (S.Clave, S.Descripcion, S.Precio);
```

4. Dependencias

Librería	Versión	Uso
PyQt6	6.5+	Interfaz gráfica (widgets, hilos, señales)
pandas	1.5+	Lectura/procesamiento Excel/CSV
pyodbc	4.0+	Conexión ODBC a SQL Server
openpyxl	3.0+	Motor lectura .xlsx
reportlab	3.6+	Generación PDFs
Python	3.10+	Lenguaje base

```
pip install PyQt6 pandas pyodbc openpyxl reportlab
```

5. Módulos y Clases Principales

5.1 pantalla_herrajes.py

Clase	Tipo	Descripción
PantallaHerrajes	QMainWindow	Ventana principal, lógica UI
ImportDialog	QDialog	Configuración importación (hojas, mapeo)
ImportWorker	QObject	Worker asíncrono en QThread
UpperLineEdit	QLineEdit	Fuerza mayúsculas
NavLineEdit	UpperLineEdit	Enter = Tab

5.2 db_manager.py

- **listar_todos()** → SELECT * ORDER BY Clave
- **buscar(texto)** → LIKE en Clave y Descripcion
- **insertar_o_actualizar(reg)** → UPSERT simple
- **bulk_upsert_max_precio(lista)** → MERGE masivo con tabla temporal

5.3 Parsers de Importación

Parser	Proveedor	Características
parse_excel_herralum.py	Herralum	Auto-detección encabezados, mapeo configurable, cálculo precio unitario (PZ,CJ
parse_excel_profilo.py	Profilo	Columnas fijas (2,3,5), limpieza precios ($, comas)
parse_csv.py	Genérico	CSV: clave, descripcion, precio

5.4 ventana_divisora.py

QDialog que se abre durante la importación para registros con precio_invalido_o_cero. Fórmula: **Precio Unitario = Precio Original / Cantidad Divisora**

5.5 imprimir.py

Usa **ReportLab** para generar PDF tamaño Carta con encabezados repetidos, columnas CLAVE/DESCRIPCIÓN/PRECIO y pie numerado.

5.6 utils.py

- **dec2(valor)** → Decimal con 2 decimales
- **detectar_columnas_excel(df, headers)** → Auto-detección por nombre y scores de contenido

6. Flujo de Importación

[Imagen pendiente: flujo_importacion.png]

Secuencia:

1. 1. Usuario selecciona archivo (QFileDialog)
2. 2. Si Excel → ImportDialog (hojas + mapeo columnas)
3. 3. Se crea ImportWorker → QThread
4. 4. Sniff proveedor por valores UM
5. 5. Parser → (validos, rechazados)
6. 6. Deduplicación origen (Counter, colapso precio máx)
7. 7. bulk_upsert_max_precio() → MERGE SQL
8. 8. Si rechazados precio_invalido_o_cero → VentanaDivisora
9. 9. Resultados ventana → insertar_o_actualizar()
10. 10. Mensaje final con resumen + RL pendientes

7. Hilos y Concurrencia

```
self._import_thread = QThread(self)
self._import_worker = ImportWorker(...)
self._import_worker.moveToThread(self._import_thread)

self._import_thread.started.connect(self._import_worker.run)
self._import_worker.progress.connect(progress.setValue)
self._import_worker.status.connect(progress.setLabelText)
self._import_worker.finished.connect(_ok)
self._import_worker.error.connect(_fail)

self._import_thread.start()
```

La UI se bloquea con **_set_controles_habilitados(False)** y se libera en **_cleanup()**.

8. Manejo de Errores

Nivel	Mecanismo	Ejemplo
Validación UI	QDoubleValidator, comprobaciones	Precio inválido → QMessageBox.warning
Parsers	try/except por registro	precio_invalido_o_cero, clave_larga
BD	try/except operaciones	Error conexión → QMessageBox.critical
Worker	error.emit(str)	Excepción en hilo → UI
Logs	CSV rechazados	D:\vah\logs\rechazados_*.csv

9. Configuración y Personalización

Conexión (db_manager.py):

```
server = r"DESKTOP-AE60PIU\SQLEXPRESS"
database = "herrajes"
table = "dbo.materiales_herraje"
DRIVER={ODBC Driver 18 for SQL Server};
Trusted_Connection=yes;
```

Mapeo columnas persistente:

```
# D:\vah\logs\herrajes_column_mapping.json
{
  "archivo.xlsx": {
    "sheets": ["Hoja1"],
    "mapping": {"clave":"Parte","descripcion":"Descripción",...}
  }
}
```

— Fin del Manual Técnico —

Documento generado el 13/05/2026 | Versión 1.0
VAH ERP © Todos los derechos reservados